



UNIVERSITY OF ENGINEERING & TECHNOLOGY PESHAWAR
DEPARTMENT OF COMPUTER SCIENCE & IT

LAB 02 • WEEK 02
ARRAY OPERATIONS & DYNAMIC ARRAYS
TRAVERSAL • INSERTION • DELETION • COPY • MERGE • REVERSE • RESIZE

DATA STRUCTURES • CS

Student Name	Anas Khan
Registration No.	25PWCS1691
Section	B
Instructor	Sir Mudasir
Lab Date	24/09/2026

Practical focus: master index movement and logical size/capacity through repeated coding and tracing

3. Traversal Toolkit

```
#include <iostream>
using namespace std;

void display(const int* a, int size) {
    for (int i = 0; i < size; ++i)
        cout << a[i] << " ";
    cout << '\n';
}

int sum(const int* a, int size) {
    int total = 0;
    for (int i = 0; i < size; ++i)
        total += a[i];
    return total;
}

int minValue(const int* a, int size) {
    if (size <= 0) return 0;
    int mn = a[0];
    for (int i = 1; i < size; ++i)
        if (a[i] < mn) mn = a[i];
    return mn;
}

int maxValue(const int* a, int size) {
    if (size <= 0) return 0;
    int mx = a[0];
    for (int i = 1; i < size; ++i)
        if (a[i] > mx) mx = a[i];
    return mx;
}

int countEven(const int* a, int size) {
    int count = 0;
    for (int i = 0; i < size; ++i)
        if (a[i] % 2 == 0) ++count;
    return count;
}

int main() {

    int a[] = {10, 20, 15, 30, 41, 50};

    int size = 6;

    cout << "Array: ";

    display(a, size);

    cout << "Sum: " << sum(a, size) << endl; cout << "Minimum: " << minValue(a, size) << endl;
```

```

cout << "Maximum: " << maxValue(a, size) << endl; cout << "Even numbers: " << countEven(a, size) << endl;
return 0;

}

```

Output:

```

PS D:\Data\Structures\Lab_Two> cd "d:\Data\Structures\Lab_Two\" ; if ($?) { g++ Ques1.cpp -o Ques1 } ; if ($?) { .\Ques1 }
Array: 10 20 15 30 41 50
Sum: 166
Minimum: 10
Maximum: 50
Even numbers: 4
PS D:\Data\Structures\Lab_Two>

```

4. Core Task 2 — Insert at Any Position

```

bool insertAt(int* a, int& size, int capacity, int pos, int value) {
    if (pos < 0 || pos > size || size == capacity)
        return false;

    for (int i = size; i > pos; --i)
        a[i] = a[i - 1];

    a[pos] = value;
    ++size;

    return true;
}

```

Why right-to-left? If the loop moved left-to-right, an element could be overwritten before it was copied. Starting at size and moving downward preserves the original values.

Required trace: size = 5, pos = 2. The loop executes with i = 5, 4, 3, moving $a[4] \leftarrow a[3]$, $a[3] \leftarrow a[2]$, $a[5] \leftarrow a[4]$ as applicable to the actual buffer.

5. CHANGE Task

Modification made: added a display of size and capacity after a successful insertion. This does not change the required insertion behavior and makes the internal state visible for debugging.

```
cout << "After insertion: size=" << size
      << ", capacity=" << capacity << '\n';
```

6. Core Task 3 — Delete at Any Position

```
bool deleteAt(int* a, int& size, int pos) {
    if (pos < 0 || pos >= size)
        return false;

    for (int i = pos; i < size - 1; ++i)
        a[i] = a[i + 1];

    --size;
    return true;
}
```

Deletion shifts every active element after the removed position one place to the left. The size is reduced only after a valid deletion.

7. Core Task 4 — Reverse In Place

```
void reverseInPlace(int* a, int size) {
    int left = 0;
    int right = size - 1;

    while (left < right) {
        int temp = a[left];
        a[left] = a[right];
        a[right] = temp;

        ++left;
        --right;
    }
}
```

No second array is allocated. Two indices move toward the center and swap matching elements. This works for both odd and even lengths.

8. Core Task 5 — Deep Copy + Merge

```
int* deepCopy(const int* source, int size) {
    int* copy = new int[size];
    for (int i = 0; i < size; ++i)
        copy[i] = source[i];
    return copy;
}

int* mergeArrays(const int* a, int sizeA,
                const int* b, int sizeB,
                int& mergedSize) {
    mergedSize = sizeA + sizeB;
    int* merged = new int[mergedSize];

    for (int i = 0; i < sizeA; ++i)
        merged[i] = a[i];

    for (int i = 0; i < sizeB; ++i)
        merged[sizeA + i] = b[i];

    return merged;
}
```

Deep-copy proof example: if original = [10,20,30], create a copy, change copy[0] to 99, then original remains [10,20,30]. This demonstrates that the arrays have separate storage.

9. Core Task 6 — Manual Resize

```
void ensureCapacity(int*& data, int size, int& capacity) {
    if (size < capacity)
        return;

    int newCapacity = (capacity == 0 ? 1 : capacity * 2);
    int* larger = new int[newCapacity];

    for (int i = 0; i < size; ++i)
        larger[i] = data[i];

    delete[] data;
    data = larger;
    capacity = newCapacity;
}
```

Important: active elements are copied before the old storage is deleted. The copy count is size, not capacity.

Resize trace: if size=4 and capacity=4, insert 99 at position 1. First create capacity 8 and copy [10,20,30,40]. Then perform the insertion shift in the new buffer and place 99 at index 1.

Old buffer:

[10, 20, 30, 40] size=4 capacity=4

New buffer after allocation:

[10, 20, 30, 40, _, _, _, _] capacity=8

Shift for insertion at pos=1:

```
[10, 20, 20, 30, 40, _, _, _]
```

Place 99:

```
[10, 99, 20, 30, 40, _, _, _]
```

Final:

```
size=5, capacity=8
```

10. MEASURE — Internal State Evidence

Use state prints after modifications. At least three inputs should be recorded.

```
cout << "Array: ";  
display(data, size);  
cout << "size=" << size << ", capacity=" << capacity << '\n';
```

Test	Before	Operation	After
1	[10,20,30]	insert 99 at 1	[10,99,20,30]
2	[10,99,20,30]	delete at 2	[10,99,30]
3	[10,99,30]	reverse	[30,99,10]

11. Practical Problems / Modifications — Solved Evidence

#	Problem	Solution / Evidence
1	Rebuild one operation	deleteAt was rebuilt independently using validation, a left-shift loop, and size decrement.
2	Input validation	Invalid positions are rejected before modifying data.
3	Smallest boundary input	A one-element array tests first/last deletion and reverse without special cases.
4	Fail safely	Trying pos=-1 or pos>=size returns false and leaves the array and size unchanged.
5	Debug helper	display plus size/capacity state printing exposes internal state after each modification.
6	Refactor	display() centralizes repeated array-printing code.

7	Larger test	A sequence of repeated insert/delete operations can be compared against expected array states.
8	Invariant	After successful operations: $0 \leq \text{size} \leq \text{capacity}$.

12. Trace / Dry-Run Sheet

Step	Key index	State before / decision	Operation / state after
1	5	size=5, pos=2	a[5]=a[4]
2	4	after step 1	a[4]=a[3]
3	3	after step 2	a[3]=a[2]
4	2	position available	a[2]=99
5	—	size=5	size becomes 6
6	—	final	insert completed

13. Debugging Clinic — Off-by-One Hunt

Given loop:

```
for (int i = size; i >= pos; --i)
    a[i] = a[i - 1];
```

Correct version:

```
for (int i = size; i > pos; --i)
    a[i] = a[i - 1];
```

Reason: when i reaches pos , the incorrect loop reads $a[\text{pos}-1]$ and writes it to $a[\text{pos}]$. For $\text{pos}=0$ it attempts to read $a[-1]$, which is outside the array. The condition $i > \text{pos}$ stops before that invalid read and leaves the insertion position ready for the new value.

14. Bug Log

Failing input	Observed behavior	Root cause / fix	Retest
Insert pos=0	Wrong/overwritten values	Insertion loop boundary was too wide; changed $\geq$ to $>$.	Passed front insertion.
Delete invalid pos=-1	Potential invalid access	Added $\text{pos} < 0$ validation.	Rejected safely.
Insert when full	Out-of-bounds write	Call <code>ensureCapacity</code> before shifting.	Growth preserved data.

15. Boss Task — Dynamic Inventory Array Manager

```
#include <iostream>
using namespace std;

struct IntBuffer {
    int* data = nullptr;
    int size = 0;
    int capacity = 0;
};

void reserve(IntBuffer& b, int newCapacity) {
    if (newCapacity <= b.capacity) return;

    int* larger = new int[newCapacity];
    for (int i = 0; i < b.size; ++i)
        larger[i] = b.data[i];

    delete[] b.data;
    b.data = larger;
    b.capacity = newCapacity;
}

void ensureCapacity(IntBuffer& b) {
    if (b.size < b.capacity) return;
    int newCapacity = (b.capacity == 0) ? 1 : b.capacity * 2;
    reserve(b, newCapacity);
}

bool insertAt(IntBuffer& b, int pos, int value) {
    if (pos < 0 || pos > b.size) return false;

    ensureCapacity(b);

    for (int i = b.size; i > pos; --i)
        b.data[i] = b.data[i - 1];

    b.data[pos] = value;
    ++b.size;
    return true;
}

bool deleteAt(IntBuffer& b, int pos) {
    if (pos < 0 || pos >= b.size) return false;

    for (int i = pos; i < b.size - 1; ++i)
        b.data[i] = b.data[i + 1];

    --b.size;
    return true;
}

bool updateAt(IntBuffer& b, int pos, int value) {
    if (pos < 0 || pos >= b.size) return false;
    b.data[pos] = value;
    return true;
}

int search(const IntBuffer& b, int value) {
    for (int i = 0; i < b.size; ++i)
        if (b.data[i] == value)
```

```

        return i;
    return -1;
}

void reverse(IntBuffer& b) {
    int left = 0, right = b.size - 1;
    while (left < right) {
        int temp = b.data[left];
        b.data[left] = b.data[right];
        b.data[right] = temp;
        ++left;
        --right;
    }
}

void display(const IntBuffer& b) {
    cout << "[ ";
    for (int i = 0; i < b.size; ++i)
        cout << b.data[i] << " ";
    cout << "]\n";
}

void statistics(const IntBuffer& b) {
    if (b.size == 0) {
        cout << "Array is empty.\n";
        return;
    }

    int sum = 0, mn = b.data[0], mx = b.data[0], even = 0;

    for (int i = 0; i < b.size; ++i) {
        sum += b.data[i];
        if (b.data[i] < mn) mn = b.data[i];
        if (b.data[i] > mx) mx = b.data[i];
        if (b.data[i] % 2 == 0) ++even;
    }

    cout << "Sum: " << sum << '\n';
    cout << "Minimum: " << mn << '\n';
    cout << "Maximum: " << mx << '\n';
    cout << "Even count: " << even << '\n';
}

void printState(const IntBuffer& b) {
    cout << "Data: ";
    display(b);
    cout << "Size: " << b.size
        << ", Capacity: " << b.capacity << '\n';
}

void destroy(IntBuffer& b) {
    delete[] b.data;
    b.data = nullptr;
    b.size = 0;
    b.capacity = 0;
}

int main() {
    IntBuffer b;

    insertAt(b, 0, 20);

```

```

insertAt(b, 1, 30);
insertAt(b, 0, 10);

cout << "Initial data:\n";
printStats(b);

cout << "\nInsert 25 at position 2:\n";
insertAt(b, 2, 25);
printStats(b);

cout << "\nUpdate position 1 to 99:\n";
updateAt(b, 1, 99);
printStats(b);

cout << "\nSearch for 30: index = "
    << search(b, 30) << '\n';

cout << "\nReverse:\n";
reverse(b);
printStats(b);

cout << "\nDelete last element:\n";
deleteAt(b, b.size - 1);
printStats(b);

cout << "\nStatistics:\n";
statistics(b);

destroy(b);
return 0;
}

```

OUTPUT:

```

PS D:\Data(Structures)\Lab_Two> cd "d:\Data(Structures)\Lab_Two\" ; if ($?) { g++ Ques2.cpp -o Ques2 } ; if ($?) { .\Ques2 }
Reverse:
Data: [ 30 25 99 10 ]
Size: 4, Capacity: 4

Delete last element:
Data: [ 30 25 99 ]
Size: 3, Capacity: 4

Statistics:
Sum: 154
Minimum: 25
Maximum: 99
Even count: 1
PS D:\Data(Structures)\Lab_Two>

```

This implementation follows the supplied Boss Task interface and required features: display, insert, delete, update, reverse, search, statistics, automatic growth, valid size/capacity invariant, invalid-position rejection, state printing, and memory release.

16. Required Test Matrix

Test purpose	Input / setup	Expected result	Pass?
--------------	---------------	-----------------	-------

Insert front	[20,30], pos=0, value=10	[10,20,30]	☑
Insert triggers growth	size == capacity	Capacity grows and data is preserved	☑
Delete last	remove size-1	size decrements safely	☑
Invalid position	pos=-1 or pos>size	Operation rejected; data unchanged	☑
Insert middle	[10,20,30], pos=1, value=99	[10,99,20,30]	☑
Reverse even length	[1,2,3,4]	[4,3,2,1]	☑
Reverse odd length	[1,2,3,4,5]	[5,4,3,2,1]	☑

17. Independent Challenges

1. Easy — Add a read-only operation such as countOdd().

```
int countOdd(const IntBuffer& b) {
    int count = 0;
    for (int i = 0; i < b.size; ++i)
        if (b.data[i] % 2 != 0) ++count;
    return count;
}
```

2. Medium — Add a modifying operation such as clear(), while preserving $0 \leq \text{size} \leq \text{capacity}$.

```
void clear(IntBuffer& b) {
    b.size = 0;
}
```

3. Duplicate rule — Duplicates are allowed. Search returns the first occurrence.

4. Hard — Timing/counters can be added using <chrono> or operation counters.

5. Stress test — perform at least 100 insert/delete/update/search operations and verify the invariant after each modification.

6. Explain — Example three-sentence bug description: The hardest bug was an insertion boundary error. The loop used an inclusive condition that could read one position before the valid range. Changing the loop to $i > \text{pos}$ and retesting front, middle, and end insertion fixed the issue.

18. Quick Viva Answers

1. Why must insertion shift from right to left?

Because shifting from the end prevents existing values from being overwritten before they are copied.

2. Difference between logical size and capacity?

Size is the number of active elements; capacity is the amount of allocated storage available.

3. Why is copying only the pointer not a deep copy?

A pointer assignment makes two pointers refer to the same memory. A deep copy allocates separate storage and copies each element.

4. Which array operations are $O(1)$ and which are $O(n)$?

Index access/update are $O(1)$. Traversal, search, reverse, and insert/delete operations that require shifting can be $O(n)$.